

About

This lab lets you practice writing boolean expressions, **if-elif-else** statements, and transforming control flow diagrams into code.

Learning Objectives

1. Practice writing boolean expressions and **if-elif-else** statements
2. Practice understanding and transforming control flow diagrams into code.

Estimated Size

1 program file.

Setup

In your Python files, you must include author information. On the first line, add a **# Author comment line**, where you mention your full name. Similarly, include **your email and Spire ID** on the **# Email** and **# Spire ID** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

[Download the starter code file `triangle\_class.py`](#) and open it in VSCode. **All your work** should be in this single file, and this is the file you will submit to Gradescope. **Do NOT change the file name**. First, please **modify the comments section at the top of the file** to include your information.

0. Triangles and Classifications

You will be implementing three functions in this assignment. For this lab, a triangle is defined by the lengths of its three edges **a**, **b**, **c**, and we assume they are all **positive integers** and **sorted** such that **a ≤ b ≤ c**. In general, the three edges certainly don't have to be integers and don't need to be sorted, but for this lab, we make these assumptions for simplicity. A triangle can be classified in two ways: by angles and by edge lengths. Your job is to implement a function that classifies triangles based on edge lengths as detailed below, by using **if-elif-else** statements and boolean expressions.

1. Implement `is_edge_sorted` (7 points)

Since we assume the three edges **a**, **b**, and **c** must be sorted, let's first write a function to check whether this condition is **True**. Write a function, called `is_edge_sorted`, which should:

- Take the three edges **a**, **b**, **c** as parameters
- Return **boolean True** if **a ≤ b** and **b ≤ c**; otherwise, return **boolean False**

Note that the return type of your function must be **boolean**, it may NOT be a string. Some useful tips:

- For this function, there is no need to use if-else: you can just return a boolean expression
- Python allows you to **chain** comparison operators, for example, you can write `a<=b<=c<=d` and that's equivalent to `a<=b and b<=c and c<=d`

The starter code contains example code which you can uncomment to help **test your function**. Follow the instructions there. Below are some examples of running a correct implementation of this function:

```
print(is_edge_sorted(3, 4, 5))
True
print(is_edge_sorted(3, 5, 3))
False
```

For the following question, you can assume the three edges are sorted, and the autograder will always use sorted edges as input to test your functions.

2. Implement `classify_by_edges` (7 points)

A triangle can be classified as **equilateral**, **scalene**, or **isosceles** by its edges. An **equilateral** triangle is one where all three edges are equal; an **isosceles** triangle has two edges that are equal but not all three; a **scalene** triangle is one where all three edges are different (i.e., no two edges are equal).

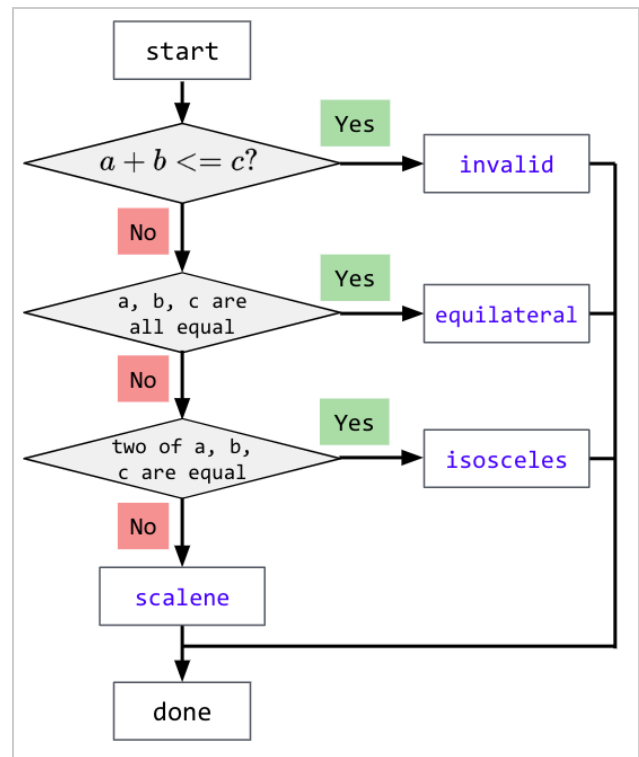
However, before checking those conditions, you should first check the validity of the triangle: if the sum of the two shorter edges `a+b` is less than or equal to the longest edge `c`, this is invalid, as you would not be able to connect the three edges to make a triangle.

The control flow diagram on the right shows how the classification is done. It assumes `c` is the longest edge of the triangle (which is true by our assumption).

Your task is to write a function, called `classify_by_edges`, which should:

- Take the three edges `a`, `b`, and `c` as parameters
- Return a **string** representing the triangle class. The string must be one of `invalid`, `equilateral`, `isosceles` and `scalene`, and it may NOT be anything else.

This question lets you practice writing boolean expressions using logical operators. Think about how to



implement each condition in the diagram.

3. Implement `classify_by_angles` (1 point)

A triangle can be classified as **right**, **obtuse**, or **acute** by its angles. A **right** triangle is one where one of the angles is exactly 90 degrees. An **obtuse** triangle has one angle that is greater than 90 degrees, while an **acute** triangle is one where all three angles are less than 90 degrees.

However, before checking those conditions, you should first check the validity of the triangle: if the sum of the two shorter edges `a+b` is less than or equal to the longest edge `c`, this is invalid, as you would not be able to connect the three edges to make a triangle. To classify a triangle by its angles based on the lengths of its sides, we can leverage the relationship between the sides of a triangle and its angles. Assuming `c` is the longest edge of the triangle (which is true by our assumption). For valid triangles, we can use the Pythagorean theorem to identify right triangles: if the square of `c` equals the sum of the squares of `a` and `b`, the triangle is **right**. If the square of `c` is greater than the sum of the squares of `a` and `b`, the triangle is **obtuse**. Otherwise, if the square of `c` is smaller, the triangle is **acute**.

Your task is to write a function, called `classify_by_angles`, which should:

- Take the three edges `a`, `b`, and `c` as parameters
- Return a **string** representing the triangle class. The string must be one of `invalid`, `right`, `obtuse`, or `acute`, and it may NOT be anything else.

4. Submit your code to Gradescope

Log in to [Gradescope](#), select **Lab 03: Code**, and submit `triangle_class.py`. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

5. Submit Lab Questionnaire on Gradescope

Remember to submit the Lab 3 questionnaire to Gradescope, too.

If you see a question you do not know how to answer, ask TAs/UCAs in your lab, use Piazza, or go to one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the

course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.